

Assignment Template: Project Milestone 1

Team Name: Project Healthify

Team Members: Hans Anstey, Yogi Chopra, Atticus Crook & Mohamed Dawoud

Course Code: CPTS 322

Date: Sep 24th, 2026 - Sep 28th, 2026

1. Tools and Technologies

IDEs: Visual Studio Code

- We will use Visual Studio Code as our primary integrated development environment. Visual Studio Code is a great choice for our project, since it is free and cross-platform, essential for our team who will be working across different operating systems. It is also attractive for its numerous and easily connected integrations for Git. Additionally, it provides various free extensions for coding in Python, very important to us as our project will be written almost entirely in Python. However, we do not plan to limit the scope of the project to only allow for Visual Studio Code as the primary IDE, and other IDEs (even the terminal) should be able to be used to successfully edit and run our application.

Version Control System: Git with GitHub as our forge

- Using Git with GitHub is a good version control system for the project as it supports the scrum process model. Allowing continuous development, testing and integrating of the changes incrementally. Git will provide branching, merging, commit history, and rollback ability, making it easy to implement new features without disrupting the main branch. GitHub has pull requests, code reviews, and issue tracking. GitHub integrates easily with Pytest and Docker builds, allowing them to be incorporated into the CI/CD pipelines. Using branch protection and limiting repository permissions can improve security and further code quality. Git and GitHub will be able to provide collaboration, traceability, and the integration needed to begin Project Healthify.

Framework: Reflex, written in Python, compiled into React components

- For our application, we wanted to use a framework everyone on our team would feel comfortable learning in a short time frame that still allowed us to create a sophisticated application. We landed on a framework called Reflex, which uses Python, a relatively easy language to learn (and one well suited for working with the web API's we will need

to use for our project) for styling components that Reflex can compile into React components for our frontend, and uses Starlette under the hood for page routing and handling HTTPS requests to external APIs. This framework will help us learn the basics of web development (typically in a Next.js project) using components for UI, custom HTTP endpoints, and connecting with database instances to create a full-stack web application.

Libraries: Reflex[db] (SQLModel, Alembic & Pydantic), Python-dotenv, Psycopg[binary] Httpx, Bcrypt, Cryptography, Pytest

- We will use several Python libraries to implement our full-stack web application, which can be sorted into five general categories: framework, database management, HTTPS requests, encryption, and testing. Our framework for both our frontend and backend is Reflex, which will compile our Python code in React Components which can be rendered in the browser. We will also use Python-dotenv for tracking our local environment variables (e.g. wearable provider OAuth, database URL, encryption keys, etc.). Included in the Reflex[db] library are the SQLModel, Alembic and Pydantic libraries, which are used for defining our SQL table schemas, tracking schema migrations and validating data types. Then we will use Httpx (included with Reflex) for integrating third-party API's with HTTPS requests (such as MyPlate's BMI calculator or potentially USDA's food look-up API). We plan to do simple one-way hashing encryption for our passwords and authentication for connecting with our wearable provider and will need the bcrypt and cryptography libraries to do so. Finally, we plan to use Pytest for testing.

Client/Server Technologies: React.js with maybe a REST API and a WebSockets link

- Our client-server architecture will center on React.js to drive a dynamic, component-based user interface, linked to our Python backend through a combination of a REST API and WebSockets. React.js will handle the frontend presentation, allowing users to interact with their health dashboards and visualize metrics smoothly. To retrieve static data, standard REST APIs will manage stateless transactions such as securely authenticating users, updating profile demographics, and downloading historical health reports. However, because REST's request-response model is inefficient for live data

streaming, we will also incorporate a WebSockets link. This persistent, bidirectional connection between the React frontend and the FastAPI backend will instantly push continuous wearable telemetry and critical anomaly alerts directly to the user's dashboard. This dual-protocol approach perfectly balances secure, scalable data retrieval with the low-latency demands of real-time health monitoring.

Programming Language: Python

- Python is a good programming language as it is able to provide flexibility and the large library needed for developing the server-side components, database integration, testing, and supporting functionality. Python works well with FastAPI, SQLAlchemy, and Pytest, allowing for smoother development of the project. Its usability and readability make it easy to maintain and understand the existing code in the project. The large existing system for Python means it already has libraries for authentication, data validation, API development, database access, and automated testing. Python can also be packed into Docker containers, making deployment across development extremely easy for all people on the project.

Deployment Tool: Docker

- Docker is a good deployment tool for the project as it is able to package the application and its dependencies into a standardized container that can run consistently across environments. Docker can isolate application components, ensuring developers use compatible versions of the software. Docker also supports scalability as its application is able to create more containers when more capacity is required. Security can be improved within Docker by creating permissions and using minimal images. It integrates very easily with GitHub, allowing the project to build and test the same container environment repeatedly, decreasing development time and increasing throughput of the project.

Testing framework: Pytest

- For our testing infrastructure, we have selected Pytest as our primary testing framework. Pytest is widely regarded as the industry standard for Python applications due to its highly readable syntax and minimal boilerplate requirements. By utilizing standard

Python assertion statements, Pytest allows our team to write clear and concise unit tests for our data ingestion pipelines and threshold alerting systems without learning a complex testing API. Furthermore, Pytest's powerful fixtures and mocking capabilities are essential for the HealthTrack project. They enable us to reliably simulate continuous wearable device data streams, isolate our FastAPI endpoints, and mock database transactions without requiring a live connection. This ensures every functional requirement is continuously validated against our Gherkin acceptance scenarios, promoting a highly stable codebase.

Database Technology: PostgreSQL

- We will implement PostgreSQL as our core backend server database technology to data storage required by HealthTrack. PostgreSQL is a powerful, open-source relational database renowned for its strict data integrity, advanced security controls, and exceptional scalability. Because our application must securely compartmentalize sensitive healthcare profiles and data, along with continuous time-series telemetry, PostgreSQL's robust indexing and role-based access policies are invaluable. Additionally, PostgreSQL provides excellent support for JSONB data types, giving our team the flexibility to store unstructured wearable payload data alongside highly structured relational tables. Its ability to partition massive datasets ensures that query performance remains consistently fast as historical health data accumulates, providing a highly reliable and secure foundation.

2. Software Process Model: Scrum

We will use the Scrum software process model for the development of HealthTrack. Scrum is an Agile framework that organizes development into short, time-limited iterations called sprints, allowing a project to be developed incrementally. At the beginning of each sprint, the team selects a set of requirements or user stories to implement. The team then designs, develops, tests, and integrates those features during the sprint, producing a usable increment of the system. At the end of each sprint, the completed work can be reviewed, and the results can be used to determine what should be developed or improved in the following sprint.

Scrum fits HealthTrack particularly well because the project is expected to be completed through three sprints during the semester. Instead of attempting to develop the entire health-monitoring system at once, the team can progressively build its functionality. For example, the first sprint could establish the project foundation, database, device connection, and basic user interface. The second sprint could implement health-data management, more functionality in the dashboards, and related backend functionality. The final sprint could focus on real-time monitoring, alerts, testing, integration, and deployment. This is only an example. This allows each sprint to produce a meaningful increment while keeping the overall project manageable.

The project also has some uncertainty and technical risk because it involves multiple components, including a React frontend, FastAPI backend, PostgreSQL database, wearable data, REST APIs, WebSockets, Docker, and automated testing. Scrum allows the team to discover integration or technical problems earlier rather than waiting until the end of development. However, Scrum also requires effective communication and careful sprint planning. With only three sprints, poor prioritization could result in important features being left until the end. Therefore, the team should prioritize the core functionality first and maintain a realistic backlog for each sprint.

Overall, Scrum provides a structured but flexible approach that matches the project's three-sprint timeline, incremental deliverables, technical complexity, and need for continuous testing and integration.